CSS326 Lab Assignment #9: Transactions

Student ID: 6722782164

Q1

In Part A I reduced book 1's stock by 2 and added an order, then used COMMIT, so both changes were saved. In Part B I ran the same statements with ROLLBACK, and the data looked exactly the same as before Part B, so nothing was saved.

id	title	stock
1	Database System Concepts	10
NULL	NULL	NULL

id	customer_id	book_id	quantity
1	1	1	1
2	2	4	2
3	3	2	1
NULL	NULL	NULL	NULL

PART A

id	title	stock
1	Database System Concepts	8
NULL	NULL	NULL

id	customer_id	book_id	quantity
1	1	1	1
2	2	4	2
3	3	2	1
4	4	1	2
NULL	NULL	NULL	NULL

PART B

id	title	stock
1	Database System Concepts	8
NULL	NULL	NULL

id	customer_id	book_id	quantity
1	1	1	1
2	2	4	2
3	3	2	1
4	4	1	2
NULL	NULL	NULL	NULL

Q2

The route first checks that book_id and quantity are positive whole numbers, otherwise it returns 400. Then it gets one connection, starts a transaction, lowers the stock, inserts the order, and commits with a 201 reply.

Q3

The stock update only works when stock is enough (stock >= quantity). If not, it rolls back and returns 409, and no order is made. Here book 5 has 1 copy and I ordered 3, so the stock stayed 1.

id	title	stock
1	Database System Concepts	10
2	Learning SQL	5
3	Node.js Design Patterns	3
4	Clean Code	8
5	Eloquent JavaScript	1
6	Designing Data-Intensive Apps	0

```
### Q3 - not enough stock (book 5 has only 1 copy) -> expect 409
Send Request
POST http://localhost:3000/customers/2/orders
Content-Type: application/json
```

Response(16ms)

```
HTTP/1.1 409 Conflict
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 48
ETag: W/"30-2pzFwlb0Cwh007cgYMdFM+9Pz00"
Date: Thu, 22 Oct 2026 00:52:00 GMT
Connection: close

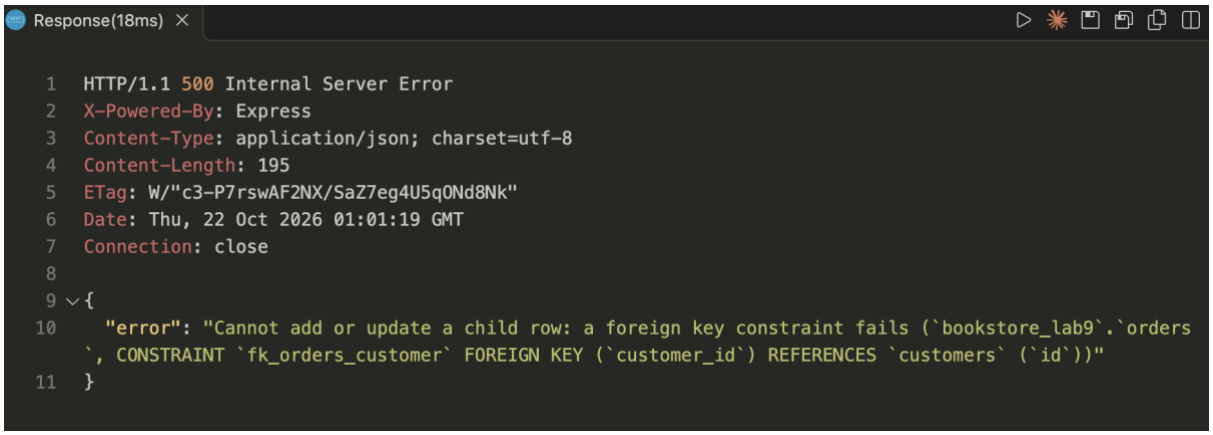
{
  "error": "not enough stock (or book not found)"
}
```

Q4

I ordered as customer 999, who does not exist, so the foreign key rejects the INSERT after the stock was already reduced. The code rolled back, and book 1's stock was 10 before and still 10 after.

id	title	stock
1	Database System Concepts	10
2	Learning SQL	5
3	Node.js Design Patterns	3
4	Clean Code	8
5	Eloquent JavaScript	1
6	Designing Data-Intensive Apps	0





Q5

In one transaction, the route finds the order, deletes it, and adds the quantity back to the book's stock. If the order does not exist for that customer, it rolls back and returns 404.

Q6

Book 3 started with stock 3. After ordering 2 copies the stock was 1, and after cancelling that order it went back to 3.

Before order

id	title	stock
1	Database System Concepts	10
2	Learning SQL	5
3	Node.js Design Patterns	3
4	Clean Code	8
5	Eloquent JavaScript	1
6	Designing Data-Intensive Apps	0

Customer Ordered

```
POST http://localhost:3000/customers/2/orders
Content-Type: application/json

{ "book_id": 3, "quantity": 2 }
```

Response(11ms)

```
HTTP/1.1 201 Created
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 24
ETag: W/"18-GoCLkomXwyOkCToN9ZZFk0VZeLE"
Date: Thu, 22 Oct 2026 01:04:15 GMT
Connection: close

{
  "ok": true,
  "order_id": 5
}
```

	3	Node.js Design Patterns	1

Cancel order

```
DELETE http://localhost:3000/customers/2/orders/5
```

Response(11ms)

```
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 11
ETag: W/"b-Ai2R8hgEarLmHKwesT1qcY913ys"
Date: Thu, 22 Oct 2026 01:10:50 GMT
Connection: close

{
  "ok": true
}
```

	3	Node.js Design Patterns	3	

Q7

Atomicity means a set of steps either all happen or none of them happen. For example, in a bank transfer, taking money from one account and adding it to another must both succeed, or the money could be lost.

Q8

A transaction only works on one connection. If I use the pool directly, each query may use a different connection, so the rollback would not undo the earlier queries.